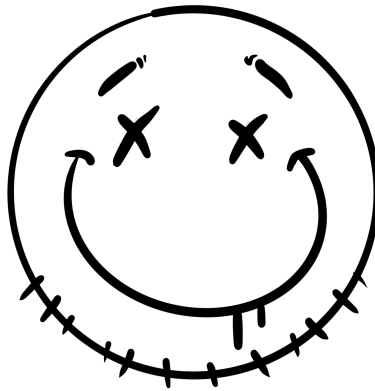




Lab 11: Malware Analysis

Keith Schoen

CSEC3755-001

Dr. Daniel Pittman

30 April 2026

Contents

1	Executive Summary	1
2	Triage & File Identification	1
2.1	Creating the EICAR Test File (Sample A)	1
2.2	Basic File Identification	2
2.3	Hash Lookup on Virus Total	2
3	Static Analysis	3
3.1	String Analysis with Sysinternals Strings	3
3.2	Examine PE Structure via VirusTotal's Details Tab	5
3.3	Choose a Real Malware Sample (Sample B)	5
3.3.1	Exploring Sample B's Virus Total Page	6
4	Dynamic Analysis	9
4.1	Analyzing the Execution Timeline	9
4.1.1	Process Activity	9
4.1.2	File System Activity	9
4.1.3	Registry Activity	10
4.1.4	Network Activity	10
4.2	Connecting to L24 Concepts	10
4.3	Map to MITRE ATT&CK	13
5	IOC Report & Detection	14

1 Executive Summary

2 Triage & File Identification

Note about this section:

The Add-WindowsCapability command for Sysmon that was given did not work for this environment.

Instead, I used the following:

```
# Download

Invoke-WebRequest -Uri "https://download.sysinternals.com/files/Sysmon.zip" -OutFile "$env:
TEMP\Sysmon.zip"

# Extract

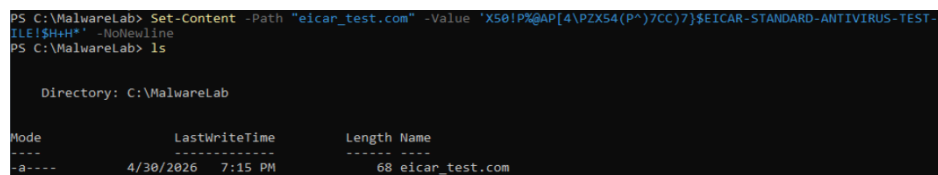
Expand-Archive -Path "$env:TEMP\Sysmon.zip" -DestinationPath "C:\Tools\Sysmon"

# Install (run this as Admin)

C:\Tools\Sysmon\Sysmon64.exe -accepteula -i
```

2.1 Creating the EICAR Test File (Sample A)

The following screenshots show the process of creating the EICAR test file in the Windows VM. I used the full sting given hoping to see Defender defy my expectations and catch the file at creation, but the file survived create and Get-Content

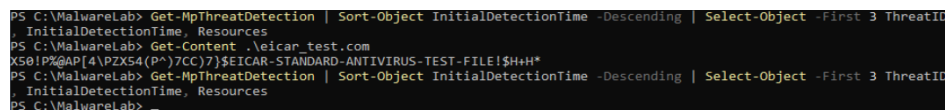


```
PS C:\MalwareLab> Set-Content -Path "eicar_test.com" -Value 'X50!P%0AP[4\pZX54(P^)7CC)7)$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+h*' -NoNewline
PS C:\MalwareLab> ls

Directory: C:\MalwareLab

Mode                LastWriteTime         Length Name
----                -
-a----           4/30/2026   7:15 PM             68 eicar_test.com
```

Figure 1.1: Creation of the EICAR test file



```
PS C:\MalwareLab> Get-MpThreatDetection | Sort-Object InitialDetectionTime -Descending | Select-Object -First 3 ThreatID, InitialDetectionTime, Resources
PS C:\MalwareLab> Get-Content .\eicar_test.com
X50!P%0AP[4\pZX54(P^)7CC)7)$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+h*
PS C:\MalwareLab> Get-MpThreatDetection | Sort-Object InitialDetectionTime -Descending | Select-Object -First 3 ThreatID, InitialDetectionTime, Resources
PS C:\MalwareLab>
```

Figure 1.2: The EICAR test file survives Windows Defender

2.2 Basic File Identification

```

PS C:\MalwareLab> (Get-Item .\eicar_test.com).Length
68
PS C:\MalwareLab> Get-FileHash -Algorithm SHA256 .\eicar_test.com

Algorithm      Hash
-----
SHA256         5B726692BEFC9CE6B935B8081A1161F969DADD8DD921299416910EB3A8A1D702
Path
----
C:\MalwareLab\eicar_test.com

PS C:\MalwareLab> Get-FileHash -Algorithm MD5 .\eicar_test.com

Algorithm      Hash
-----
MD5            6A26CC095F9132549AE21EA4065899F6
Path
----
C:\MalwareLab\eicar_test.com

PS C:\MalwareLab> Get-Content .\eicar_test.com
X50!P%AP[4\PZX54(P^7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
PS C:\MalwareLab> strings.exe .\eicar_test.com

Strings v2.54 - Search for ANSI and Unicode strings in binary images.
Copyright (C) 1999-2021 Mark Russinovich
Sysinternals - www.sysinternals.com

X50!P%AP[4\PZX54(P^7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*
PS C:\MalwareLab> _

```

Figure 1.3: Results of file identification commands for the EICAR test file

2.3 Hash Lookup on Virus Total

- Out of 62 security vendors on Virus Total, 6 flagged the file as malicious.
- A few names given for the threat are:
 - Virus/EICAR_Test_File
 - NotAThreat.EICAR[TestFile]
 - Eicar_test_1
- Tags shown for the 'threat' are:
 - powershell
 - idle
 - long-sleeps
- File type according to Virus Total: **Powershell**
- As for the community tab, there isn't any activity early then 10 years.

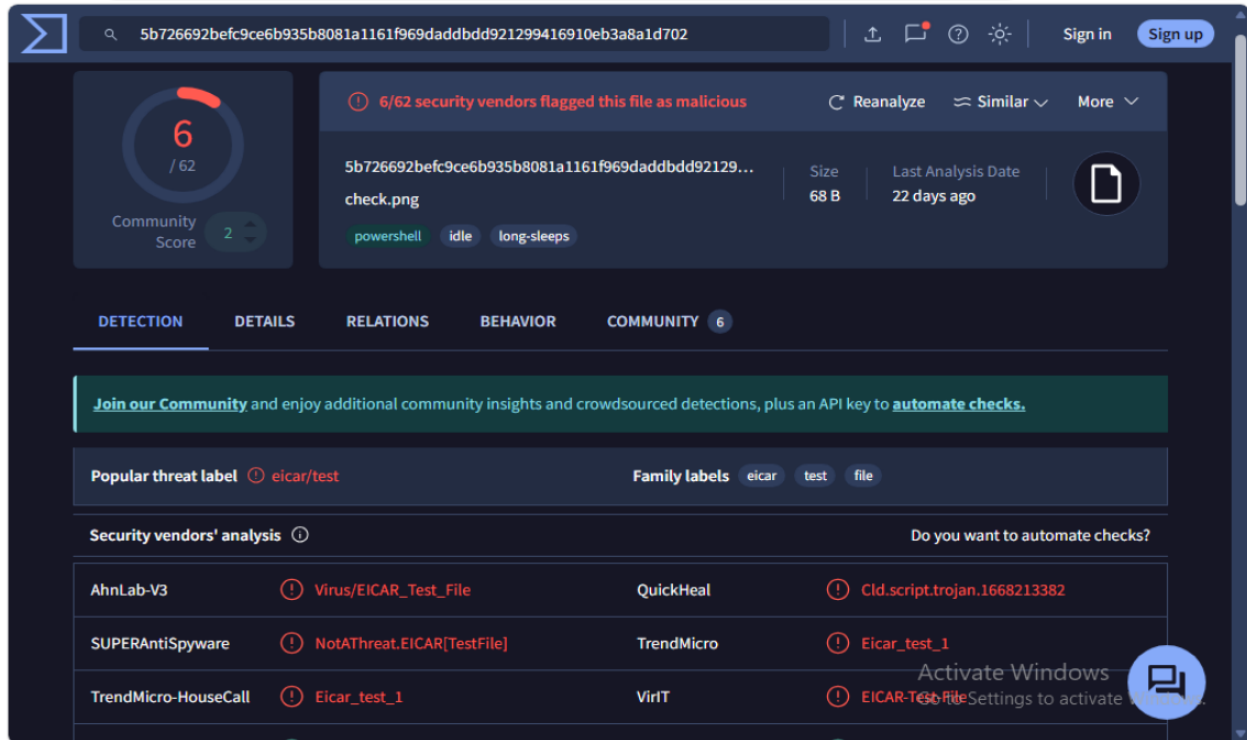


Figure 1.4: Results for Virus Total hash look-up

It should be noted that while the results on Virus Total showed a 6/62, the only results to show in the 'Security vendors' analysis' table were those from AhnLab-V3, SUPERAntiSpyware, and TrendMicro-HouseCall.

What kind of threat is this?

This is an idle **Powershell script** that waits until the right conditions are met before executing. This would explain how it was able to evade detection from Windows Defender despite being flagged as malicious.

3 Static Analysis

3.1 String Analysis with Sysinternals Strings

Executed the following commands on the EICAR test file from section one to demonstrate how it works at a basic level:

```
# Extract all strings at least 8 characters long and save to file
strings.exe -n 8 .\eicar_test.com > eicar_strings.txt
```

```
# Open the output and examine it  
notepad eicar_strings.txt
```

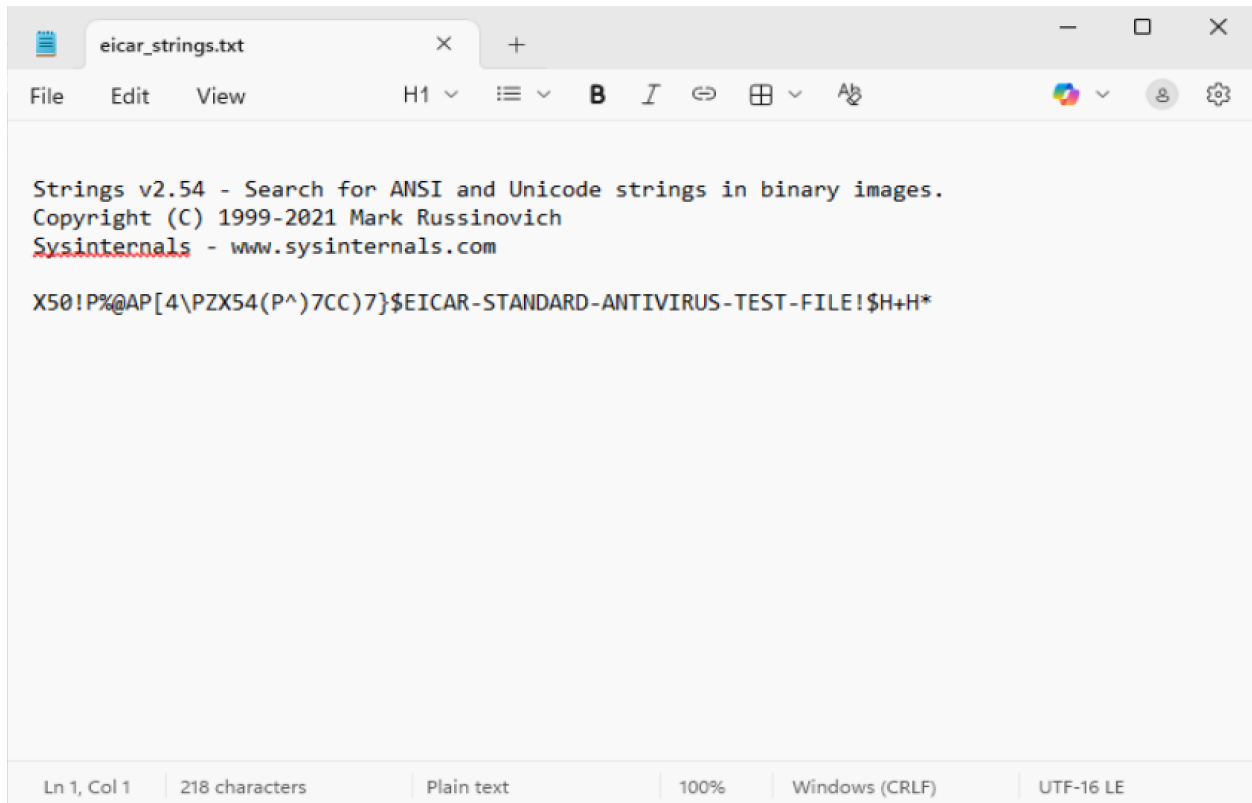


Figure 2.1: Strings output for the EICAR test file

3.2 Examine PE Structure via VirusTotal's Details Tab

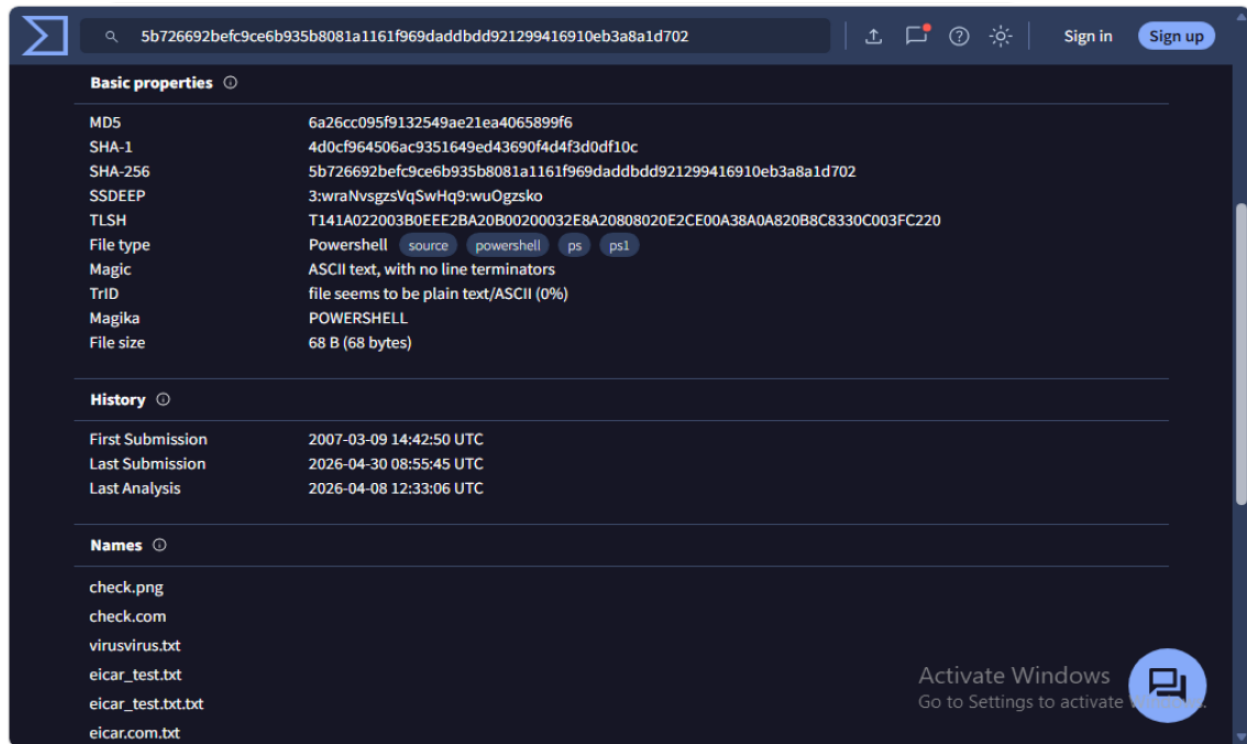


Figure 2.2: Details section for EICAR test file hash lookup

3.3 Choose a Real Malware Sample (Sample B)

Note about malware selection for Sample B - XWORM

SHA256 Hash: 939b247d1cf5a8b674ff632365af9982256363d6ae390876d0d3d9cbc07d163b

Malware Family: phantomstealer

First submission to VT: 2026-04-30 07:14:26 UTC

VT Report URL:

<https://www.virustotal.com/gui/file/939b247d1cf5a8b674ff632365af9982256363d6ae390876d0d3d9cbc07d163b/summary>

ANY.RUN Report URL:

<https://any.run/report/939b247d1cf5a8b674ff632365af9982256363d6ae390876d0d3d9cbc07d163b/45507e5c-bc86-405f-bc18-2b912cf28181>

Honestly, I decided on a RAT because I like the animal and the malware category. When I did quick overview, this one had tons of documentation making it an easy first grab. Along with the amount of

information already becoming available, it is made extra exciting in that this is brand-new submission to VT (about 3 hours old). It's a great pick for Sample B.

3.3.1 Exploring Sample B's Virus Total Page

Detection Tab

- **Detection Ratio:** 39/71
- **XWORM**
- **Family:** phantomstealer

Details Tab

- **File type and size:** Win32 EXE / 876.50 KB (897536 bytes)
- **First Submission:** 2026-04-30 07:14:26 UTC
- **PE Header Information:**
 - Target Machine: Intel 386 or later processors and compatible processors
 - Compilation Timestamp: 2026-04-30 06:12:58 UTC
 - Entry Point: 902118
 - Contained Sections: 3
- **Imports:**
 - mscoree.dll
 - * _CorExeMain

Behavior Tab

- **Sandboxes:**
 - CAPA
 - VirusTotal Observer
- **Process Tree:**
 - 840 - 939b247d1cf5a8b674ff632365af9982256363d6ae390876d0d3d9cbc07d163b.exe
 - 276 - C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe Add-MpPreference -ExclusionPath C:\Users\j\USER\Downloads\939b247d1cf5a8b674ff632365af9982256363d

6ae390876d0d3d9cbc07d163b.exe

2124 - ead8bccff0d208d26935d05c0ec80515.bat

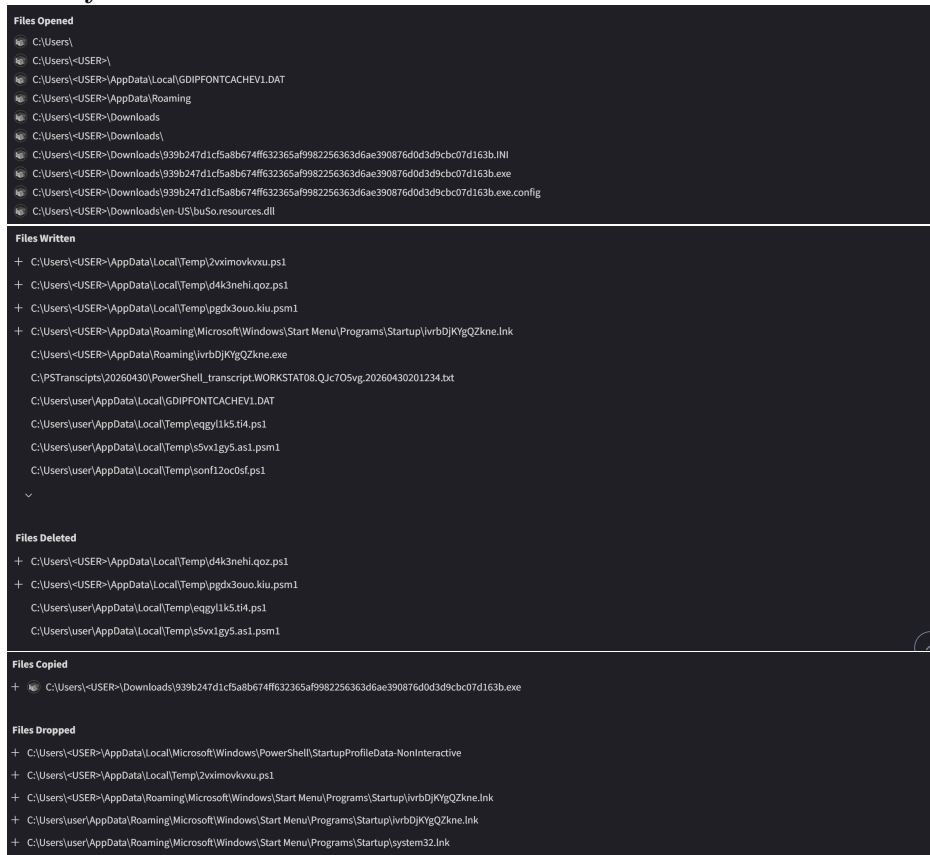
2932 - ead8bccff0d208d26935d05c0ec80515.bat

2968 - 0906e49c252cde958db247fef205b1bb.exe

892 - cde09bcd5fde1e2eac52c0f93362b79.exe

2860 - 0e058c51d287f1a00f674d8fefdf1e35.exe

- **File System Actions:**



Figures 2.3-.5: Details of Malware file operations

- **Registry Actions:**

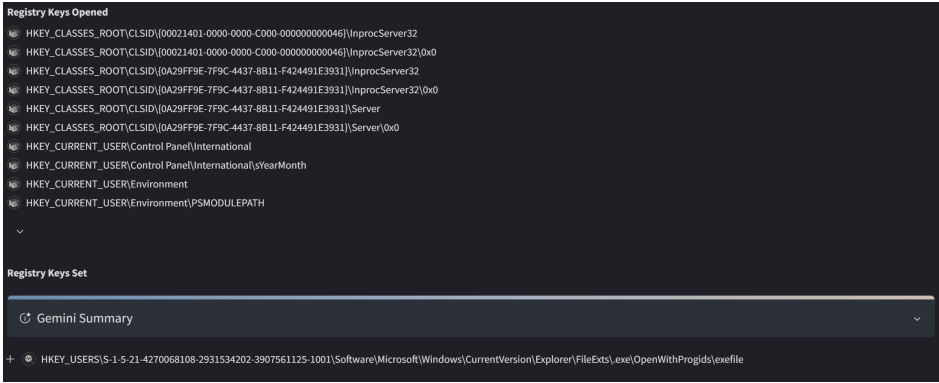


Figure 2.6: Details of Malware registry actions

• Network Communications

– IP Traffic

* UDP 8.8.8.8:53

* TCP 199.232.210.172:80

• MITRE ATT&CK Techniques:

Execution	Persistence	Privilege Escalation	Stealth	Credential Access
TA0002 4 Techniques	TA0003 5 Techniques	TA0004 3 Techniques	TA0005 6 Techniques	TA0006 2 Techniques
Scheduled Task/Job T1053	New Service T1050	New Service T1050	Obfuscated Files or Information T1027	Input Capture T1056
Command and Scripting Interface T1059	Scheduled Task/Job T1053	Scheduled Task/Job T1053	Deobfuscate/Decode Files or Information T1140	Hooking T1179
PowerShell T1086	Registry Run Keys / Startup Folder T1060	Hooking T1179	Hide Artifacts T1564	
Shared Modules T1129	Modify Registry T1112		Reflective Code Loading T1620	
	Hooking T1179			

Figure 2.7: Table of MITRE ATT&CK techniques used

Relations Tab

• Contacted IPs:

IP	Detections	Autonomous System	Country
199.232.210.172	0 / 91	54113	US
8.8.8.8	0 / 91	15169	US

• Dropped Files:

Scanned	Detections	File type	Name
2026-04-30	0 / 62	Windows shortcut	system32.lnk
2026-04-30	21 / 62	Windows shortcut	lvrbDJYgQZkne.lnk
2026-04-30	39 / 71	Win32 EXE	TT-PAYMENT COPY -04.bat
2026-04-30	0 / 61	?	StartupProfileData-NonInteractive
?	?	file	03b338a80e7e7e9d3b2543d19fcd557de85664ae7500efd2cf086120f4d8b0
?	?	file	0d5b7d1dc823364ff41d6fcafe9f76b9e1290ebbf387a3b6a2ff4c655a76c
?	?	file	0f835269f16770370277ad215a8c50cf7492a4a3265a7e08cf07ebba2fbeb31e
?	?	file	445c2baeb30c50743737052b1880c3c413be0b2f2ce790c9dd449835d8e8400b
?	?	file	56acc3b428c2561797056a2c367ed5e8501aba2262bb4030cb6706c719abda5f
?	?	file	7267c26e6calf9dd018a0eb8cc2b2fbf41957d8c716cb7873d812f4d8f741a

• Related Samples:

Execution Parents (2) ⓘ			
Scanned	Detections	Type	Name
2026-04-30	19 / 62	Email	Temp[217].eml
2026-04-30	39 / 71	Win32 EXE	TT-PAYMENT COPY -04.bat
Bundled Files (4) ⓘ			
Scanned	Detections	File type	Name
2026-04-28	0 / 61	XML	.rsrc/MANIFEST/1
?	?	file	.reloc
?	?	file	.rsrc/version.bat
?	?	file	.text

4 Dynamic Analysis

Information from this section was obtained from the report created at the ANY.RUN link given at the beginning of section 3.

4.1 Analyzing the Execution Timeline

4.1.1 Process Activity

- The application launches itself via .exe file
- The initial application launches multiple child processes (exe.exe (#XWORM), schtasks.exe, and con-host.exe)
- Injected into explorer.exe, svchost.exe, and system32.exe
- For each process needed to gather information to send back the the C2, exe.exe runs schtasks.exe to run legitimate windows applications (such as PDF FILE PREVIEWER) for future tasks.

4.1.2 File System Activity

- This malware did not create any new files.
- Five executables were dropped

```

— C:\Users\admin\AppData\Local\Temp\p3zwhafveq1.ps1
— C:\Users\admin\AppData\Roaming\system32.exe

```

- C:\Users\admin\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\system32.lnk
- C:\Users\admin\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\ivrbDjKYgQZkne.lnk
- C:\Users\admin\AppData\Roaming\ivrbDjKYgQZkne.exe
- Rewrote C:\Users\admin\AppData\Roaming\system32.exe
- There were no **delete events**.

4.1.3 Registry Activity

There was one modification event that showed evidence of persistence:

- **PID Process:** (6856) exe.exe
- **Operation:** Write
- **Value:** C:\Users\admin\AppData\Roaming\system32.exe
- **Key:** HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
- **Name:** system32

4.1.4 Network Activity

- All network activity was whitelisted and looked legitimate. However, there is one malicious connection:
 - **PID** 6956
 - **PROCESS** exe.exe
 - **IP** 158.94.211.33:7017
 - **DOMAIN** N/A
 - **ASN** OMEGATECH-AS
 - **CN** SC
 - **REPUTATION** Malicious
- 'Whitelisted' activity included HTTP requests and connections to DELL and Microsoft services, and DNS requests to Microsoft and Google services.
- Activity strongly indicates **command and control** activity meaning all data could potentially be extracted.

4.2 Connecting to L24 Concepts

The following table maps each observed behavior from the ANY.RUN sandbox report of Sample B (SHA256: 939b247d...d163b, identified as **XWorm RAT**) to the specific detection tools and techniques covered in

Lecture 24. Each row answers: *what did the malware do, which L24 tool would catch it, and what would that tool actually show?*

Malware Behavior	L24 Tool	What You Would See
Execution from Temp. <code>exe.exe</code> runs from <code>...\Local\Temp\</code> (PID 2676), parent: <code>explorer.exe</code> .	Process Explorer; Sysmon EID 1	<i>ProcExp</i> : <code>exe.exe</code> nested under <code>explorer.exe</code> ; Image Path shows <code>Temp\</code> ; Verified Signer: <i>(Not Verified)</i> . <i>Sysmon EID 1</i> : Image: <code>...Temp\exe.exe</code> ParentImage: <code>explorer.exe</code> IntegrityLevel: Medium Hashes: SHA256=...
Self-relaunch. <code>exe.exe</code> (PID 2676) spawns a second <code>exe.exe</code> (PID 6856); <code>system32.exe</code> repeats likewise.	Process Explorer; Sysmon EID 1	<i>ProcExp</i> : Multiple <code>exe.exe</code> entries with <code>exe.exe</code> as parent — identical parent/child binary is a hallmark of self-injection. <i>Sysmon EID 1</i> : Image = ParentImage = <code>...Temp\exe.exe</code> repeated across PIDs.
Defender exclusion via LOLBin. <code>powershell.exe</code> Add-MpPreference -ExclusionPath <code>...exe.exe</code> disables AV on the payload.	Autoruns; Sysmon EID 1	<i>Autoruns</i> : Run-key entry (<code>system32</code>) survives <i>Hide Microsoft Entries</i> because entry authorship is the malware, not Microsoft — the exact LOLBin evasion from L24. <i>Sysmon EID 1</i> : Image: <code>powershell.exe</code> CommandLine: <code>Add-MpPreference -ExclusionPath ...exe.exe</code> ParentImage: <code>...exe.exe</code>
Run-key persistence. HKCU\...\Run key <code>system32</code> → <code>AppData\Roaming\system32.exe</code> (PID 6856, write).	Autoruns; Sysmon EID 13	<i>Autoruns Logon tab</i> : Entry <code>system32</code> → <code>AppData\Roaming\system32.exe</code> ; Publisher: <i>(Not Verified)</i> ; pink-tinted row. <i>Sysmon EID 13</i> : Key: HKCU\...\Run Value: <code>...Roaming\system32.exe</code> Process: <code>exe.exe</code>

Malware Behavior	L24 Tool	What You Would See
Startup folder shortcuts. Two .lnk files dropped to Startup\:\system32.lnk and ivrbdjKYgQZkne.lnk.	Autoruns; Sysmon EID 11	<i>Autoruns Logon tab:</i> Both shortcuts listed; randomized name ivrbdjKYgQZkne.lnk immediately stands out. <i>Sysmon EID 11:</i> TargetFilename: ...\\Startup\system32.lnk Image: exe.exe
Payload dropped to AppData. system32.exe and ivrbdjKYgQZkne.exe written to AppData\Roaming to masquerade as a system binary.	Sysmon EID 11; Process Explorer	<i>Sysmon EID 11:</i> TargetFilename: ...Roaming\system32.exe Image: exe.exe <i>ProcExp:</i> When system32.exe runs, Path column shows AppData\Roaming\ not C:\Windows\System32 — L24 triage step 5 red flag.
Scheduled task (1-min interval). schtasks.exe /create /sc minute /mo 1 /tn "system32" /tr "...system32.exe" run by exe.exe (PID 4656).	Autoruns; Sysmon EID 1	<i>Autoruns Scheduled Tasks tab:</i> Task system32 → ...Roaming\system32.exe, trigger: every 1 minute. <i>Sysmon EID 1:</i> Image: schtasks.exe CommandLine: /create /sc minute /mo 1 /tn "system32" ... ParentImage: ...exe.exe
C2 beacon (XWorm TCP). exe.exe (PID 6856) opens TCP to 158.94.211.33:7017 (OMEGATECH-AS); flagged <i>Malicious</i> by Suricata.	TCPView; Sysmon EID 3	<i>TCPView:</i> exe.exe ESTABLISHED 158.94.211.33:7017 visible in real time; port 7017 is non-standard. <i>Sysmon EID 3:</i> Image: ...exe.exe DestinationIp: 158.94.211.33 DestinationPort: 7017 Protocol: tcp — attributes network traffic to the process, which tcpdump alone cannot.

Malware Behavior	L24 Tool	What You Would See
DNS connectivity check. UDP to 8.8.8.8:53 from <code>exe.exe</code> before C2 contact.	Sysmon EID 3; tcpdump	<i>Sysmon EID 3:</i> Image: <code>exe.exe</code> DestinationIp: 8.8.8.8 DestinationPort: 53 Protocol: <code>udp tcpdump: sudo tcpdump -i any -n port 53</code> shows UDP query to 8.8.8.8 from the host, providing a network-layer view independent of the host OS.
PS1 staging script dropped. <code>p3zwhafveql.ps1</code> written to <code>AppData\Local\Temp</code> by <code>exe.exe</code> (PID 2676).	Sysmon EID 11; Sysmon EID 1; Procmon	<i>Sysmon EID 11:</i> TargetFilename: <code>...Temp\p3zwhafveql.ps1</code> Image: <code>exe.exe</code> — random-name <code>.ps1</code> written by a non-PowerShell process is suspicious. <i>Sysmon EID 1</i> (on execution): Image: <code>powershell.exe</code> CommandLine: <code>-File ...p3zwhafveql.ps1</code> <i>Procmon:</i> Filter on <code>exe.exe</code> → WriteFile event shows full path to <code>.ps1</code> .

Summary: Every malicious behavior observed in this XWorm sample maps directly to at least one tool from L24. The most powerful combination is **Autoruns** (catching all three persistence mechanisms simultaneously) paired with **Sysmon** (providing full forensic provenance for each event). TCPView and Sysmon Event ID 3 together provide host-side attribution of the C2 beacon that tcpdump alone — which sees the traffic but not the responsible process — cannot supply.

4.3 Map to MITRE ATT&CK

Scheduled Task/Job (T1053) - Execution

- The malware alters scheduled tasks for recursive calls to establish persistence.
- In the case of XWORM, it registers and creates system tasks to activate background processes and establish a connection to the C2 server.

Registry Run Keys/Startup Folder (T1060) - Persistence

- The malware add at program to the startup folder or referencing it with a Register Key therefore

causing the malware to be executed whenever the user logs in.

- XWORM attempted to set its autorun on the system startup.
- This strategy belongs in the Persistence column.

Hooking (T1179) - Privilege Escalation

- The malware uses Windows built-in API functions to implant code in ordinary processes to be executed across platforms to potentially elevate privileges.
- XWORM used this function as a sort of keylogger.

Reflective Code Loading (T1620) - Stealth

- Malicious code is loaded directly into process memory, disguising it as it moves through the system, executing when it reaches its target.
- XWORM invokes .NET assembly method

```
api: System.Reflection.ConstructorInfo::Invoke
```

5 IOC Report & Detection

++report written and formatted with help from Claude (Sonnet 4.6)++